SOFTWARE ENGINEERING · CHAPTER 01

Multithreading in Java

Concurrent execution of two or more threads for maximum utilisation of the CPU: creating threads, coordinating them safely, and using the modern `java.util.concurrent` toolbox.

JDK 25 LTS **Virtual threads (JDK 21+)** **java.util.concurrent**

AGENDA

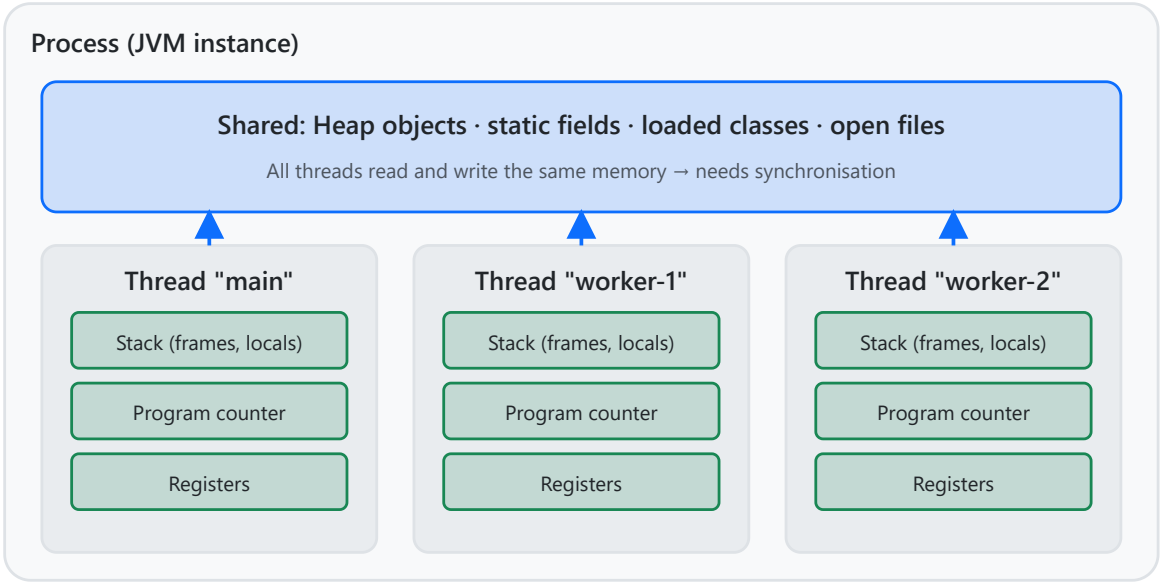
What we will cover

1. Introduction: processes, threads, concurrency
2. Anonymous class, lambda expression, and callback method
3. Creating and running threads
4. Thread lifecycle
5. Synchronization
6. Inter-thread communication
7. Thread pools and executors
8. Deadlock and thread safety
9. Thread priorities and scheduling
10. Thread interruption and handling
11. Thread-safe collections and concurrent utilities
12. Best practices for multithreading
13. Build tool primer: Apache Maven (needed for Lab-01)
14. Chapter Quiz
15. Lab-01

Click any item to jump directly to that topic.

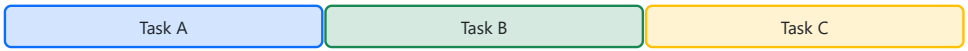
Processes, threads, concurrency

- **Process**: a running program with its own address space (the JVM is one process).
- **Thread**: an independent path of execution inside a process. Threads share the heap but each has its own *stack*, *program counter* and *registers*.
- **Concurrency**: dealing with many things at once (interleaving on one core).
- **Parallelism**: doing many things at once (several cores at the same time).
- The `main` method runs on the **main thread**; the JVM also starts GC and JIT threads.



Why use multiple threads?

1 core · sequential

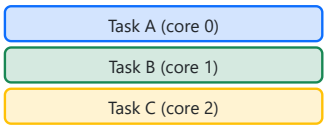


1 core · concurrent (time-sliced)



Same total CPU time, but a blocked task (I/O) no longer freezes the others → responsiveness

3 cores · parallel



3× wall-clock speed-up for CPU-bound work → throughput

Benefits	Costs
Responsive UIs and servers (never block the caller)	Race conditions and visibility bugs
Throughput on multi-core CPUs	Deadlock, livelock, starvation
Overlap I/O waiting with computation	Context-switch and memory overhead (~1 MB stack per platform thread)
Natural model for independent tasks (one request = one thread)	Hard to test and reproduce (non-deterministic)

Rule of thumb: CPU-bound → about one thread per core. I/O-bound → many more threads (or virtual threads), because most are waiting.

Anonymous class, lambda expression the building blocks

- A thread needs *something to run*: an object implementing `Runnable` (or `Callable<V>`).
- **Anonymous inner class**: declares and instantiates an unnamed class in one expression. Verbose, but can hold state and override several methods.
- **Lambda expression**: concise implementation of a *functional interface* (exactly one abstract method). `Runnable`, `Callable`, `Consumer`, `Supplier`…
- **Method reference**: `Class::method` when the lambda only delegates.
- Captured local variables must be *effectively final*.

```
// 1) Anonymous inner class
Runnable task1 = new Runnable() {
    @Override
    public void run() {
        String me = Thread.currentThread().getName();
        System.out.println("Hello from " + me);
    }
};

// 2) Lambda expression – Runnable is a @FunctionalInterface
Runnable task2 = () ->
        System.out.println("Hello from " + Thread.currentThread().getName());

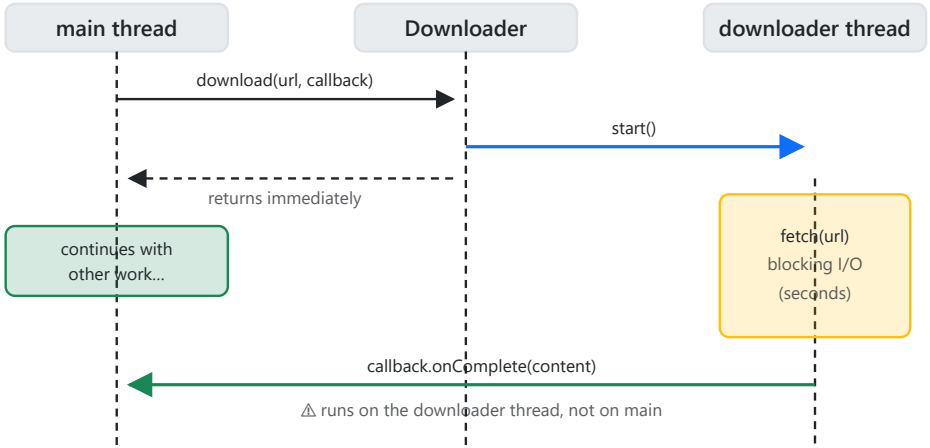
// 3) Method reference
Runnable task3 = MaintenanceJobs::cleanupTempFiles;

// 4) Callable returns a value and may throw a checked exception
Callable<Integer> task4 = () -> {
    Thread.sleep(200);
    return 42;
};

new Thread(task1, "worker-1").start();
new Thread(task2, "worker-2").start();

int limit = 10;                       // effectively final → can be captured
Runnable r = () -> System.out.println("limit=" + limit);
// limit = 20;  ✗ compile error: captured variable must be effectively final
```

Callback methods: "call me when you are done"




```java
@FunctionalInterface
interface DownloadCallback {
    void onComplete(String content);
}

class Downloader {
    void download(String url, DownloadCallback callback) {
        Thread.ofPlatform().name("downloader").start(() -> {
            String content = fetch(url);       // slow, blocking I/O
            callback.onComplete(content);      // "call back" when finished
        });
    }
    private String fetch(String url) {
        /* HTTP client … */
        return "<html>…</html>";
    }
}

// Caller does not block – it passes behaviour as a lambda
new Downloader().download("https://example.com",
        content -> System.out.println("Got " + content.length() + " chars")
);
System.out.println("Request sent, main thread continues…");
```

Watch out: the callback executes on the worker thread. Anything it touches must be thread-safe (UI toolkits usually require hopping back to their own thread).

Four ways to create a thread

```java
// A) Extend Thread – inheritance; ties task and thread together
class Worker extends Thread {
    @Override public void run() {
        System.out.println("Worker running on " + getName());
    }
}
new Worker().start();

// B) Implement Runnable – composition; preferred (task ≠ thread)
class Job implements Runnable {
    @Override public void run() {
        System.out.println("Job on " + Thread.currentThread().getName());
    }
}
new Thread(new Job(), "job-thread").start();

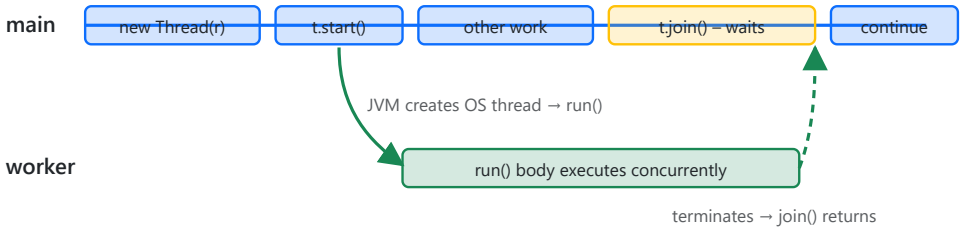
// C) Lambda
Thread t = new Thread(() -> System.out.println("Lambda thread"));
t.start();

// D) Thread builders (JDK 21+) – fluent, name patterns, platform vs virtual
Thread p = Thread.ofPlatform().name("platform-", 1).daemon(false)
                .start(() -> work());
Thread v = Thread.ofVirtual().name("virtual-", 1).start(() -> work());
ThreadFactory factory = Thread.ofVirtual().name("req-", 0).factory();
```

Approach	Pros	Cons
`extends Thread`	Simple, direct access to `getName()` etc.	No other superclass; task not reusable in pools
`implements Runnable`	Decoupled, works with executors, testable	Slightly more code
Lambda	Concise for small tasks	No state, harder to name/log
Builders	Naming patterns, virtual threads, factories	JDK 21+ only

💡 In real applications you rarely call `new Thread`. Submit tasks to an `ExecutorService` instead (topic 6).

`start()` vs `run()`, `join()`, daemon threads



t.run() ≠ t.start()

run() is a plain method call on the current thread – no new thread, no concurrency

```
public class StartVsRun {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            for (int i = 1; i <= 3; i++) {
                String me = Thread.currentThread().getName();
                System.out.println(me + " step " + i);
            }
        }, "worker");

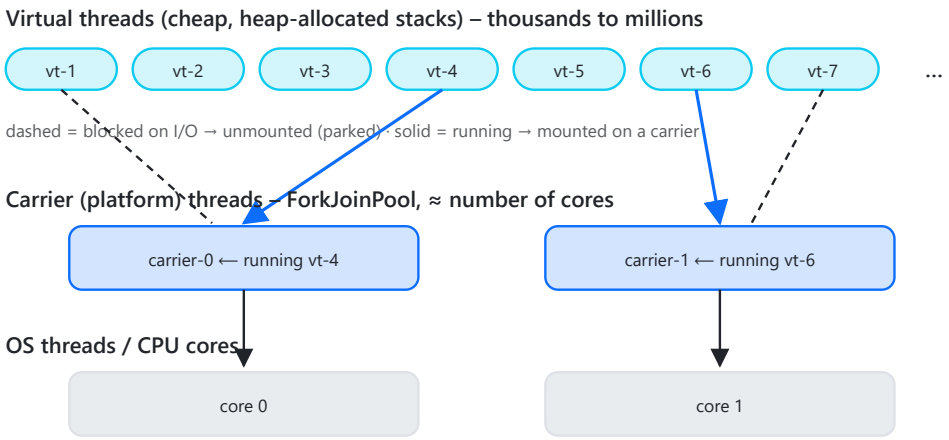
        t.run();  // ✗ prints "main step 1..3" – runs on the calling thread
        // ✓ prints "worker step 1..3" – new thread executes run()
        t.start();
        // t.start(); ✗ IllegalThreadStateException: started twice

        t.join();  // block until worker terminates (or join(timeoutMillis))
        System.out.println("worker finished, isAlive=" + t.isAlive());

        Thread bg = new Thread(() -> {
            while (true) { /* housekeeping */ }
        });
        bg.setDaemon(true);  // daemon threads don't keep the JVM alive
        bg.start();
    }   // JVM exits here: all non-daemon threads are done
}
```

- `join()` is the simplest way to wait for a result – but prefer `Future` from an executor.
- The JVM stops when the last **non-daemon** thread ends; daemon threads are killed abruptly (no `finally` guarantee).

Platform threads vs virtual threads

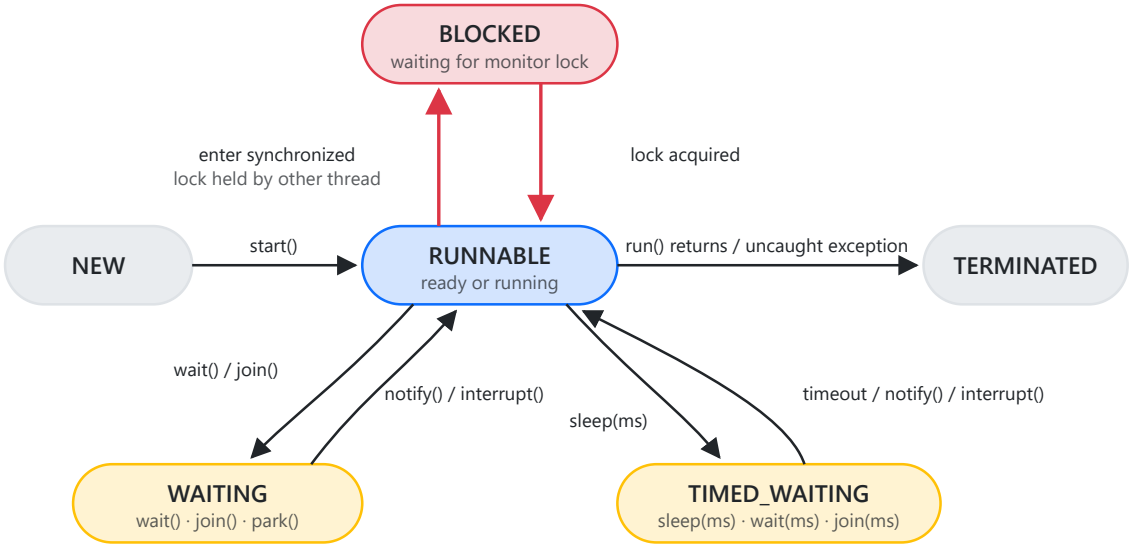



```java
// One virtual thread per task: cheap, ideal for blocking I/O at scale
try (ExecutorService executor =
             Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<String>> results = new ArrayList<>();
    for (int i = 0; i < 10_000; i++) {
        int id = i;
        results.add(executor.submit(() -> {
            // parks the virtual thread only
            Thread.sleep(Duration.ofMillis(100));
            return "order-" + id;
        }));
    }
    for (Future<String> f : results) f.get();
// close() waits for all tasks (ExecutorService is AutoCloseable since 19)
}

Thread vt = Thread.startVirtualThread(
        () -> System.out.println(
                "virtual? " + Thread.currentThread().isVirtual()));
vt.join();
```

	Platform thread	Virtual thread
Backed by	1 OS thread	JVM scheduler, mounted on carriers
Memory	~1 MB reserved stack	Grows/shrinks on heap
Best for	CPU-bound work	Many blocking tasks (I/O, requests)
Avoid	Creating thousands	Pooling them; long `synchronized` blocks pinning (much improved in JDK 24+)

Thread lifecycle & `Thread.State`



State	Meaning
`NEW`	Created, `start()` not yet called
`RUNNABLE`	Eligible to run; may be running or waiting for a CPU slice
`BLOCKED`	Waiting to acquire a monitor lock (`synchronized`)
`WAITING`	Indefinitely waiting for another thread: `wait()`, `join()`, `LockSupport.park()`
`TIMED_WAITING`	Same, with a timeout: `sleep()`, `wait(ms)`, `join(ms)`
`TERMINATED`	`run()` completed or threw; cannot be restarted

ⓘ Java does not distinguish "ready" from "running"; both are `RUNNABLE`. The OS scheduler decides which runnable thread gets a core.

Observing states with `getState()`

```
public class StateDemo {
    private static final Object LOCK = new Object();

    public static void main(String[] args) throws InterruptedException {
        Thread waiter = new Thread(() -> {
            synchronized (LOCK) {
                try { LOCK.wait(); }                    // → WAITING
                catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }, "waiter");

        Thread sleeper = new Thread(() -> {
            try { Thread.sleep(5_000); }                // → TIMED_WAITING
            catch (InterruptedException e) { /* woken early */ }
        }, "sleeper");

        System.out.println(waiter.getState());          // NEW
        waiter.start(); sleeper.start();
        Thread.sleep(100);  // give them time to block
        System.out.println(waiter.getState());          // WAITING
        System.out.println(sleeper.getState());         // TIMED_WAITING

        synchronized (LOCK) { LOCK.notifyAll(); }       // wake waiter
        sleeper.interrupt();  // wake sleeper early
        waiter.join(); sleeper.join();
        System.out.println(waiter.getState());          // TERMINATED
    }
}
```

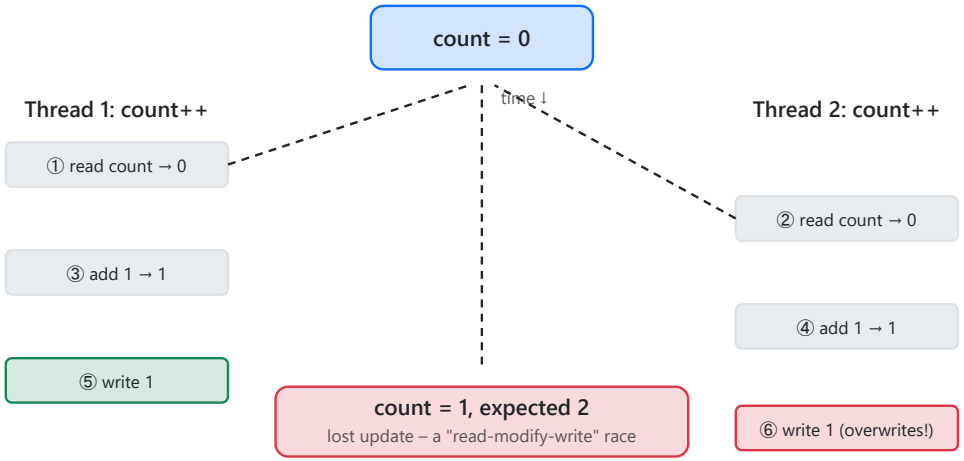
Console

```
NEW
WAITING
TIMED_WAITING
TERMINATED
```

Useful tools

- `jcmd <pid> Thread.print` or `jstack <pid>` dumps every thread with its state and stack
- `Thread.getAllStackTraces()` from inside the JVM
- JDK Mission Control / VisualVM show state timelines and lock contention
- Give threads **meaningful names**; dumps become readable

Synchronization: the problem – race conditions




```
class Counter {
    private int count = 0;
    void increment() { count++; }     // 3 bytecode steps: load, iadd, store
    int get() { return count; }
}

Counter c = new Counter();
Thread[] threads = new Thread[4];
for (int i = 0; i < threads.length; i++) {
    threads[i] = new Thread(() -> {
        for (int j = 0; j < 100_000; j++) c.increment();
    });
    threads[i].start();
}
for (Thread t : threads) t.join();

System.out.println(c.get());
// expected 400000 – actual e.g. 287341 (different every run)
```

A race condition exists when the result depends on the relative timing of threads. Two ingredients:

1. **Shared mutable state** (the `count` field), and
2. a **compound action** that is not atomic (`count++`, check-then-act).

The fixes: `synchronized`, explicit locks, atomics

```java
// 1) synchronized method – intrinsic lock on 'this' (the Class if static)
class SyncCounter {
    private int count;
    synchronized void increment() { count++; }
    synchronized int get() { return count; }  // readers must lock too!
}

// 2) synchronized block – smaller critical section, private lock object
class BlockCounter {
    private final Object lock = new Object();  // nobody else can lock it
    private int count;
    void increment() {
        synchronized (lock) { count++; }
    }
}

// 3) java.util.concurrent.locks – tryLock, timeouts, fairness, Conditions
class LockCounter {
    private final ReentrantLock lock = new ReentrantLock();
    private int count;
    void increment() {
        lock.lock();
        // ALWAYS unlock in finally
        try { count++; } finally { lock.unlock(); }
    }
}

// 4) Lock-free atomic – compare-and-swap (CAS) loop in hardware
class AtomicCounter {
    private final AtomicInteger count = new AtomicInteger();
    void increment() { count.incrementAndGet(); }
    int get() { return count.get(); }
}
```

What a lock gives you

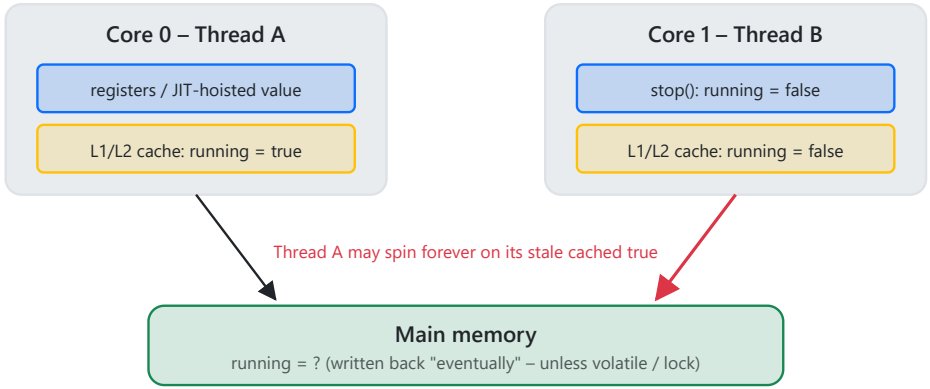
Mutual exclusion: one thread at a time in the critical section (atomicity of compound actions).

Visibility: releasing a lock flushes writes; acquiring it sees them (happens-before).

Reentrancy: the owner can re-acquire the same lock (recursive calls are safe).

Use	When
`synchronized`	Default choice: simple, JIT-optimised, auto-release
`ReentrantLock`	Need `tryLock`, timeouts, interruptible waits, fairness or multiple `Condition`s
`Atomic*`	Single variable counters/flags/references; highest throughput
`LongAdder`	Hot counters updated by many threads

Memory visibility, `volatile` and happens-before



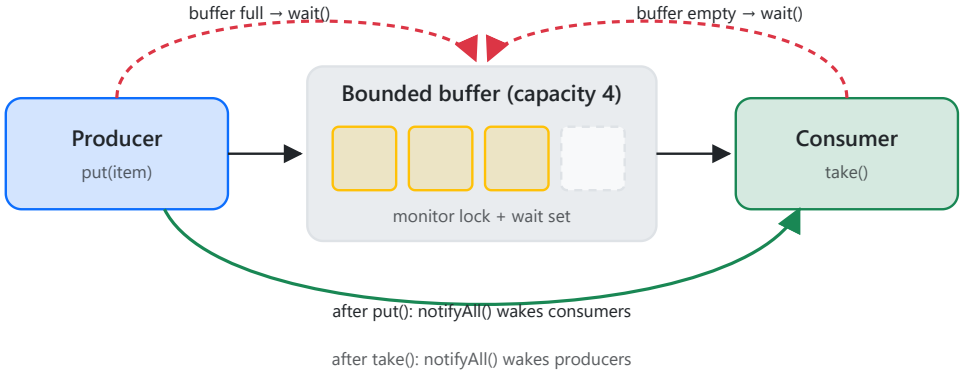

```
class Worker implements Runnable {
    // reads/writes go to main memory
    private volatile boolean running = true;

    void stop() { running = false; }           // called from another thread

    @Override public void run() {
        while (running) {  // without volatile: JIT may hoist the read out
            doWork();               // of the loop → while(true)
        }
    }
}
```

- `volatile` guarantees **visibility and ordering** for that variable, *not atomicity* of compound actions (`volatile int x; x++` is still a race).
- Java Memory Model: a write is guaranteed visible to a read only if there is a **happens-before** edge: lock release→acquire, volatile write→read, `Thread.start()`, `Thread.join()`, executor submission, `Future.get()`.
- Good fit: status flags, "publish once" references. Otherwise use `Atomic*` or a lock.

Inter-thread communication: `wait()` / `notify()`



Rules: call `wait()`/`notify()` only while holding the object's monitor; `wait()` releases the lock and re-acquires it on wake-up; always wait in a `while` loop (spurious wake-ups, multiple waiters); prefer `notifyAll()` unless you are sure one waiter is enough.

```
class BoundedBuffer<T> {
    private final Queue<T> items = new ArrayDeque<>();
    private final int capacity;
    BoundedBuffer(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        // guarded block: re-check condition
        while (items.size() == capacity) {
            wait();  // releases the monitor while waiting
        }
        items.add(item);
        notifyAll();  // state changed → wake waiting consumers
    }

    public synchronized T take() throws InterruptedException {
        while (items.isEmpty()) {
            wait();
        }
        T item = items.remove();
        notifyAll();                          // wake waiting producers
        return item;
    }
}
```

Higher-level communication: `BlockingQueue` and `Condition`

In practice, don't hand-write `wait()`/`notify()`. A `BlockingQueue` already implements the producer–consumer hand-off correctly.

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(10);
String POISON = "__STOP__";

Thread producer = new Thread(() -> {
    try {
        for (int i = 1; i <= 20; i++) {
            queue.put("msg-" + i);  // blocks while the queue is full
        }
        queue.put(POISON);  // sentinel tells consumer to stop
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
}, "producer");

Thread consumer = new Thread(() -> {
    try {
        String msg;
        // blocks while empty
        while (!(msg = queue.take()).equals(POISON)) {
            System.out.println("consumed " + msg);
        }
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
}, "consumer");

producer.start(); consumer.start();
```

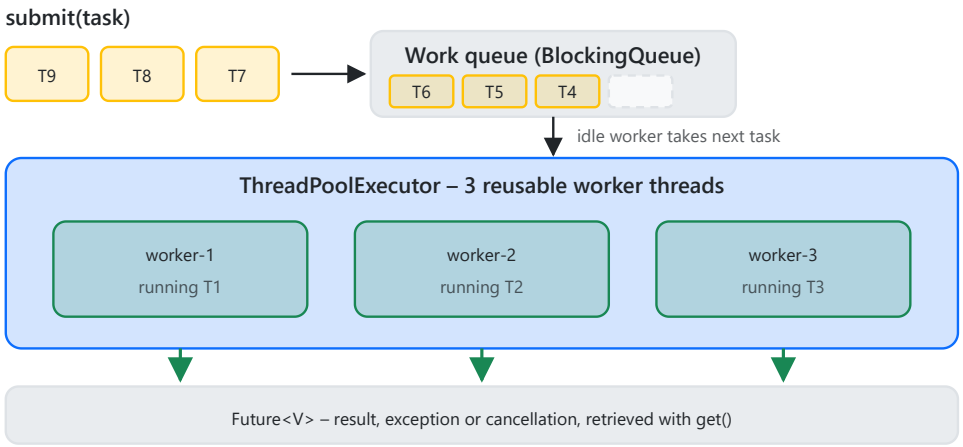
`Condition` is the `Lock`-based equivalent of `wait/notify`, with the advantage of *several* wait sets per lock (e.g. "not full" and "not empty").

```
class LockBuffer<T> {
    private final Lock lock = new ReentrantLock();
    private final Condition notFull  = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();
    private final Object[] items = new Object[16];
    private int putIdx, takeIdx, count;

    public void put(T x) throws InterruptedException {
        lock.lock();
        try {
            while (count == items.length) notFull.await();
            items[putIdx] = x;
            putIdx = (putIdx + 1) % items.length;
            count++;
            notEmpty.signal();               // wake exactly one consumer
        } finally { lock.unlock(); }
    }

    @SuppressWarnings("unchecked")
    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (count == 0) notEmpty.await();
            T x = (T) items[takeIdx];
            takeIdx = (takeIdx + 1) % items.length;
            count--;
            notFull.signal();
            return x;
        } finally { lock.unlock(); }
    }
}
```

Thread pools and the Executor framework



- **Why pool?** Thread creation is expensive; unbounded thread creation exhausts memory; pools bound concurrency and reuse threads.
- **Executor** (`execute(Runnable)`) → **ExecutorService** (`submit`, `invokeAll`, lifecycle) → **ScheduledExecutorService** (delays, periodic).

Factory (`Executors`)	Behaviour
`newFixedThreadPool(n)`	n threads, unbounded queue. CPU-bound work: n ≈ cores.
`newCachedThreadPool()`	Grows on demand, reuses idle threads (60 s). Risky under load.
`newSingleThreadExecutor()`	Sequential background processing, ordered.
`newScheduledThreadPool(n)`	Delayed / periodic tasks (replaces `Timer`).
`newVirtualThreadPerTaskExecutor()`	New virtual thread per task; ideal for I/O-heavy servers.
`newWorkStealingPool()`	ForkJoinPool; divide-and-conquer, parallel streams.

Production code usually builds `new ThreadPoolExecutor(core, max, keepAlive, unit, boundedQueue, factory, rejectionPolicy)` explicitly to control the queue size and the rejection policy.

ExecutorService, Future and CompletableFuture

```
ExecutorService pool = Executors.newFixedThreadPool(4);
try {
    Future<Integer> single = pool.submit(() -> compute(42));   // Callable
    System.out.println("result = " + single.get(2, TimeUnit.SECONDS));

    List<Callable<Integer>> tasks = List.of(
            () -> compute(1), () -> compute(2), () -> compute(3));
    for (Future<Integer> f : pool.invokeAll(tasks)) {   // waits for all
        System.out.println(f.get());  // rethrows as ExecutionException
    }
} catch (TimeoutException e) {
    System.err.println("compute(42) took too long");
} finally {
    pool.shutdown();                                    // no new tasks
    if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
        pool.shutdownNow();  // interrupt stragglers
    }
}

// Scheduled tasks
ScheduledExecutorService ses = Executors.newScheduledThreadPool(1);
ses.scheduleAtFixedRate(() -> flushMetrics(), 0, 10, TimeUnit.SECONDS);
```

```
// CompletableFuture: compose asynchronous steps without blocking
CompletableFuture<User> user =
        CompletableFuture.supplyAsync(() -> userService.load(7));
CompletableFuture<Orders> orders =
        CompletableFuture.supplyAsync(() -> orderService.load(7));

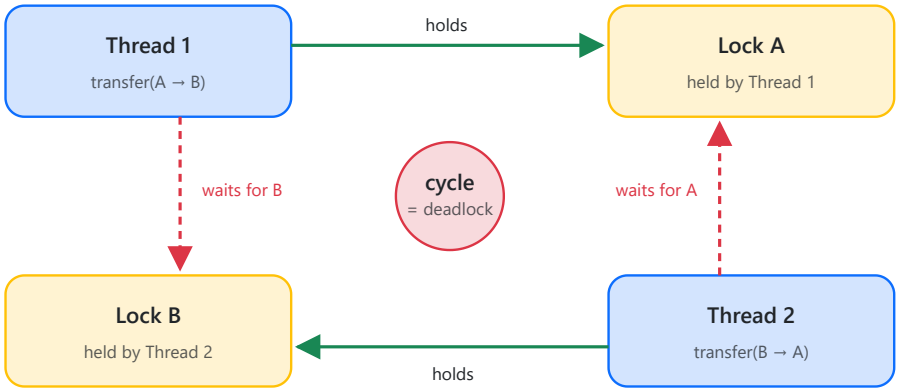
CompletableFuture<String> report = user
        .thenCombine(orders,
                     (u, o) -> u.name() + " has " + o.count() + " orders")
        .thenApply(String::toUpperCase)
        .orTimeout(3, TimeUnit.SECONDS)
        .exceptionally(ex -> "fallback: " + ex.getMessage());

report.thenAccept(System.out::println);   // callback style …
String value = report.join();             // … or block for the value

// Fan-out / fan-in
List<CompletableFuture<Integer>> futures = ids.stream()
        .map(id -> CompletableFuture.supplyAsync(() -> price(id), pool))
        .toList();
CompletableFuture.allOf(futures.toArray(CompletableFuture[]::new)).join();
int total = futures.stream().mapToInt(CompletableFuture::join).sum();
```

- Default executor for `*Async` methods is the common `ForkJoinPool`; pass your own pool for blocking work.
- Exceptions inside tasks are captured in the `Future`; they surface only when you call `get()`/`join()`. Never fire-and-forget without handling.

Deadlock: waiting for each other forever



Coffman conditions (all four needed): mutual exclusion · hold-and-wait · no preemption · circular wait. Break any one to prevent deadlock.

```
class Account {
    private int balance;
    Account(int balance) { this.balance = balance; }
    void withdraw(int amt) { balance -= amt; }
    void deposit(int amt)  { balance += amt; }
}

static void transfer(Account from, Account to, int amount) {
    synchronized (from) {              // T1 locks a … T2 locks b …
        synchronized (to) {  // T1 waits for b, T2 waits for a → stuck
            from.withdraw(amount);
            to.deposit(amount);
        }
    }
}

Account a = new Account(100), b = new Account(100);
new Thread(() -> transfer(a, b, 10), "T1").start();
new Thread(() -> transfer(b, a, 20), "T2").start();
// Both threads stay BLOCKED forever.
// jstack reports: "Found one Java-level deadlock"
```

🔍 **Diagnose** with `jstack <pid>`, `jcmd <pid> Thread.print`, or programmatically with `ManagementFactory.getThreadMXBean().findDeadlockedThreads()`.

Avoiding deadlock and designing for thread safety

```java
// 1) Global lock ordering – always acquire locks in the same order
static void transfer(Account from, Account to, int amount) {
    Account first  = from.id() < to.id() ? from : to;
    Account second = from.id() < to.id() ? to   : from;
    synchronized (first) {
        synchronized (second) {
            from.withdraw(amount);
            to.deposit(amount);
        }
    }
}

// 2) tryLock with timeout – give up and retry instead of waiting forever
boolean transfer(Account from, Account to, int amount)
        throws InterruptedException {
    while (true) {
        if (from.lock.tryLock(50, TimeUnit.MILLISECONDS)) {
            try {
                if (to.lock.tryLock(50, TimeUnit.MILLISECONDS)) {
                    try {
                        from.withdraw(amount); to.deposit(amount);
                        return true;
                    }
                    finally { to.lock.unlock(); }
                }
            } finally { from.lock.unlock(); }
        }
        // random back-off
        Thread.sleep(ThreadLocalRandom.current().nextInt(1, 10));
    }
}

// 3) Avoid the problem: immutable values + a single confined owner
record Money(long cents) {
    // no shared mutation
    Money plus(Money o) { return new Money(cents + o.cents); }
}
```

Thread-safety strategies (strongest first)

1. **Don't share**: thread confinement, `ThreadLocal`, scoped values, one owner per object.
2. **Share immutable state**: `final` fields, records, unmodifiable collections.
3. **Use thread-safe building blocks**: `ConcurrentHashMap`, atomics, executors, queues.
4. **Guard with locks**: smallest critical section, one lock per invariant, document it.

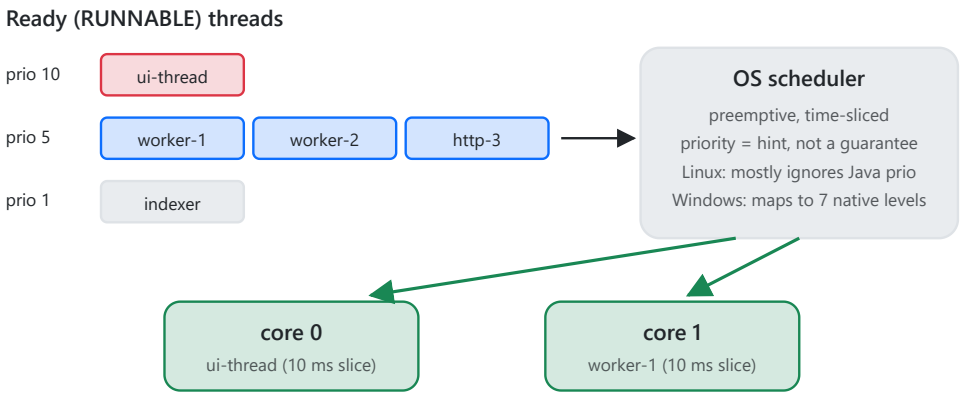
Related liveness hazards

Livelock: threads keep reacting to each other and make no progress (add random back-off).

Starvation: a thread never gets CPU or the lock (fair locks, avoid long critical sections).

Lock pinning of virtual threads by long `synchronized` blocks in older JDKs (fixed in JDK 24, JEP 491).

Thread priorities and scheduling




```
Thread low  = new Thread(() -> spin("low"));
Thread high = new Thread(() -> spin("high"));
low.setPriority(Thread.MIN_PRIORITY);      // 1
// 10  (NORM_PRIORITY = 5 is the default)
high.setPriority(Thread.MAX_PRIORITY);
low.start(); high.start();
// Linux: both get about equal CPU time. Windows: "high" finishes first.

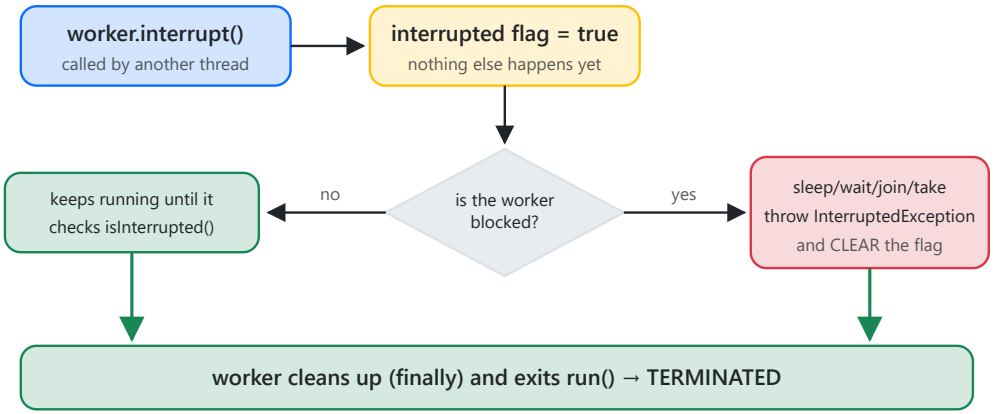
Thread.yield();  // hint: "I can give up my slice" – scheduler may ignore it

// hint inside busy-wait loops (PAUSE instruction), JDK 9+
Thread.onSpinWait();

// Virtual threads: priority is always NORM_PRIORITY; setPriority() is a no-op
```

- Priorities range 1–10 and are inherited from the creating thread.
- They are **platform-dependent hints**. Never build correctness on them – only fairness or latency tuning, and verify on the target OS.
- Real scheduling control comes from *design*: separate pools for latency-critical vs batch work, bounded queues, back-pressure.

Thread interruption: cooperative cancellation




```java
Thread worker = new Thread(() -> {
    try {
        while (!Thread.currentThread().isInterrupted()) {
            processNextChunk();  // check the flag between chunks
            Thread.sleep(200);  // blocking call: throws if interrupted
        }
    } catch (InterruptedException e) {
        // sleep() cleared the flag: restore it
        Thread.currentThread().interrupt();
    } finally {
        closeResources();                       // always runs
    }
    System.out.println("worker stopped cleanly");
}, "worker");

worker.start();
Thread.sleep(1_000);
worker.interrupt();       // a polite request, not a kill
worker.join();

// With executors: future.cancel(true) interrupts the running task;
// pool.shutdownNow() interrupts all workers and returns queued tasks.
```

Never swallow `InterruptedException` with an empty catch, and never use the removed/deprecated `Thread.stop()`/`suspend()`. Either rethrow, or restore the flag with `Thread.currentThread().interrupt()`.

Thread-safe collections

Need	Unsafe	Use instead
Map	`HashMap`	`ConcurrentHashMap` (lock striping, non-blocking reads, atomic `merge`/`compute`)
Sorted map/set	`TreeMap`	`ConcurrentSkipListMap` / `Set`
Read-mostly list/set	`ArrayList`	`CopyOnWriteArrayList` / `Set` (snapshot iteration, expensive writes)
Queue (unbounded)	`LinkedList`	`ConcurrentLinkedQueue` / `Deque` (lock-free)
Hand-off between threads	–	`ArrayBlockingQueue`, `LinkedBlockingQueue`, `PriorityBlockingQueue`, `SynchronousQueue`, `DelayQueue`
Legacy	`Vector`, `Hashtable`	Avoid: one global lock, iteration not safe

`Collections.synchronizedList(...)` wrappers make each *method* atomic, but compound actions (check-then-put, iteration) still need external locking on the wrapper.

```
ConcurrentMap<String, Integer> hits = new ConcurrentHashMap<>();
hits.merge("home", 1, Integer::sum);             // atomic read-modify-write
hits.computeIfAbsent("about", k -> 0);
hits.putIfAbsent("contact", 0);
// X NOT atomic: if (!hits.containsKey(k)) hits.put(k, 1);

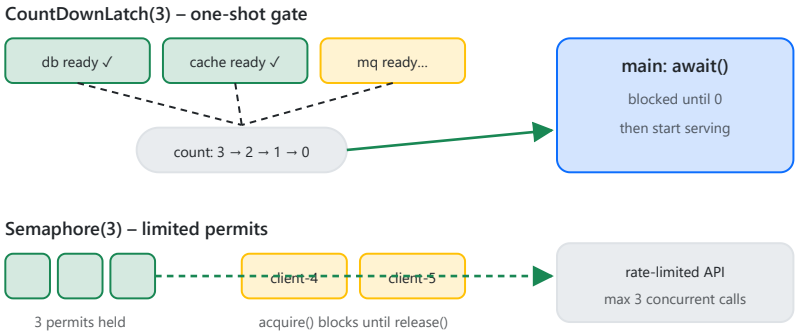
List<Listener> listeners = new CopyOnWriteArrayList<>();
for (Listener l : listeners) l.onEvent(e);  // iterates a snapshot – no CME

// lock-free, unbounded
Queue<Runnable> jobs = new ConcurrentLinkedQueue<>();
BlockingQueue<Runnable> bounded = new LinkedBlockingQueue<>(100);
bounded.offer(job, 1, TimeUnit.SECONDS);  // back-pressure with timeout

// Legacy wrapper – iteration must be manually synchronized
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
synchronized (syncList) {
    for (Integer i : syncList) { /* … */ }
}

// Immutable collections are inherently thread-safe
List<String> roles = List.of("ADMIN", "USER");
```

Concurrent utilities: latches, barriers, semaphores, locks




```
// CountDownLatch – wait until N one-time events have happened
CountDownLatch ready = new CountDownLatch(3);
for (Service s : services) {
    new Thread(() -> { s.start(); ready.countDown(); }).start();
}
ready.await();  // main proceeds when all 3 are up

// Semaphore – bound concurrent access to a resource
Semaphore permits = new Semaphore(3);
Runnable client = () -> {
    try {
        permits.acquire();
        callRateLimitedApi();
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    finally { permits.release(); }
};

// CyclicBarrier – all parties meet, run an action, then continue (reusable)
CyclicBarrier barrier =
        new CyclicBarrier(4, () -> System.out.println("phase complete"));
// inside each worker, per phase: barrier.await();

// ReadWriteLock – many concurrent readers OR one writer
ReadWriteLock rw = new ReentrantReadWriteLock();
rw.readLock().lock();
try { return cache.get(key); } finally { rw.readLock().unlock(); }

rw.writeLock().lock();
try { cache.put(key, v); } finally { rw.writeLock().unlock(); }
```

Utility	Purpose	Reusable?
`CountDownLatch`	Wait for N events	No
`CyclicBarrier` / `Phaser`	N threads rendezvous per phase	Yes
`Semaphore`	Limit concurrent users of a resource	Yes
`Exchanger`	Two threads swap objects	Yes
`StampedLock`	Optimistic reads for read-heavy data	Yes

TOPIC 11

Best practices for multithreading

1 Prefer high-level APIs

Executors, `CompletableFuture`, `BlockingQueue`, concurrent collections and parallel streams over raw `Thread`, `wait/notify` and hand-rolled locks.

2 Minimise shared mutable state

Immutability (records, `final`), confinement, and message passing eliminate whole classes of bugs. Share by communicating, don't communicate by sharing.

3 Keep critical sections small

Never call unknown code, do I/O, or sleep while holding a lock. One lock per invariant; document which lock guards which field (`@GuardedBy`).

4 Size pools deliberately

CPU-bound: ≈ cores. Blocking I/O: virtual threads or a larger pool. Use bounded queues and a rejection policy for back-pressure; name your threads.

5 Handle interruption and shutdown

Restore the interrupt flag, use `shutdown()` + `awaitTermination()`, try-with-resources on executors, and make tasks cancellable.

6 Never rely on timing or priorities

`sleep()` is not synchronisation. Use latches, futures and joins for ordering. Priorities are OS hints only.

7 Surface exceptions

Set an `UncaughtExceptionHandler`, always inspect `Future.get()`, and log failures inside `Runnable`s so tasks never die silently.

8 Test and observe

Stress tests with many threads and iterations, `jstack`/JFR for lock contention, and tools like jcstress for memory-model questions.

9 Measure before parallelising

Amdahl's law: the sequential part bounds the speed-up. Parallelism adds overhead; profile first and parallelise the proven hot spot.

Build tool primer: Apache Maven 3.9.x

Lab-01 asks you to create a *Maven project*. Chapter 02 covers build tools in depth; here is the minimum you need today.

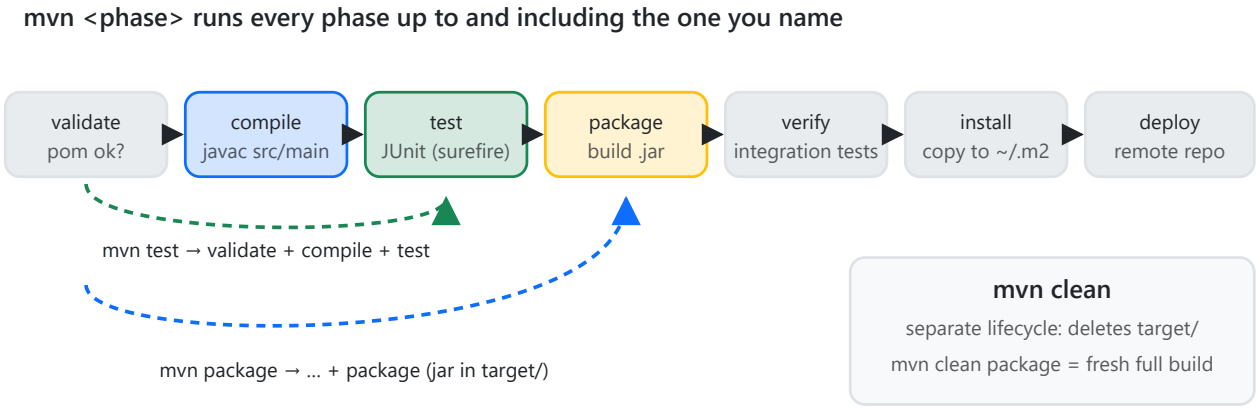
- **Why a build tool?** Real projects have hundreds of source files, external libraries (JUnit, Spring…), tests, and must produce a runnable `.jar`. Doing that with `javac` by hand does not scale and is not reproducible.
- **Maven** is the most widely used Java build tool. It is *declarative*: you describe the project in `pom.xml` (Project Object Model) and Maven knows how to compile, test and package it.
- **Convention over configuration**: put files in the standard folders and nothing needs to be configured.
- **Dependencies** are identified by *coordinates* `groupId:artifactId:version` and downloaded automatically from Maven Central into `~/.m2/repository`.
- Install from maven.apache.org (or via SDKMAN!, Homebrew, winget) and check `mvn -v`. IntelliJ IDEA, Eclipse and VS Code understand `pom.xml` directly.

Standard directory layout

```
lab01-multithreading/
├── pom.xml                  ← project descriptor
├── src/
│   ├── main/
│   │   ├── java/            ← production code
│   │   │   └── edu/itc/se/lab01/task1/Main.java
│   │   └── resources/       ← config files, go into the jar
│   └── test/
│       ├── java/            ← JUnit tests (not shipped)
│       │   └── edu/itc/se/lab01/task2/CounterTest.java
│       └── resources/
└── target/                  ← generated by Maven
    ├── classes/
    ├── test-classes/
    └── lab01-multithreading-1.0.jar
```

ⓘ The package path `edu.itc.se.lab01.task1` must match the folder path `edu/itc/se/lab01/task1/`. Never commit `target/`; add it to `.gitignore`.

The build lifecycle and the commands you will use



Command	What it does
`mvn compile`	Compile `src/main/java` into `target/classes`
`mvn test`	Compile + run all JUnit tests, report in `target/surefire-reports`
`mvn package`	… + build `target/<artifactId>-<version>.jar`
`mvn verify`	Full check used in CI and for lab submission
`mvn clean`	Delete `target/`
`mvn -q …`	Quiet: only errors and test results
`mvn -DskipTests package`	Build the jar without running tests
`mvn dependency:tree`	Show all resolved (transitive) dependencies

Running a task class after `mvn compile`

```
# compiled classes live in target/classes
java -cp target/classes edu.itc.se.lab01.task1.Main

# with program arguments (Task 1)
java -cp target/classes edu.itc.se.lab01.task1.Main --sequential

# quick experiment without Maven (single-file mode)
java src/main/java/edu/itc/se/lab01/task1/Main.java
```

Run `mvn` commands from the folder that contains `pom.xml`. The first build downloads plugins and JUnit; later builds are offline-fast.

Reading the Lab-01 `pom.xml`

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <!-- Coordinates (GAV) of YOUR project:
       groupId = organisation, artifactId = project (jar name) -->
  <groupId>edu.itc.se</groupId>
  <artifactId>lab01-multithreading</artifactId>
  <version>1.0</version>

  <properties>
    <!-- compile for JDK 25 (records, virtual threads …) -->
    <maven.compiler.release>25</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <!-- a library, by its coordinates on Maven Central -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>6.1.3</version>
      <!-- test scope: only on the test classpath -->
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Element	Meaning
`groupId` / `artifactId` / `version`	Unique identity of the artifact; the jar is named `artifactId-version.jar`
`maven.compiler.release`	Java language level and API to compile against (needs a JDK ≥ that version installed)
`<dependency>`	A library. Maven also fetches *its* dependencies (transitive)
`<scope>`	`compile` (default, everywhere) · `test` (tests only) · `provided` (server supplies it) · `runtime`
`<build><plugins>`	Optional: configure the compiler, surefire (tests), jar manifest, exec…

A first JUnit 6 test for Task 2 (`src/test/java/…/CounterTest.java`)

```java
package edu.itc.se.lab01.task2;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CounterTest {
    @Test
    void atomicCounterIsCorrect() throws InterruptedException {
        Counter c = new AtomicCounter();
        Thread[] ts = new Thread[8];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(() -> {
                for (int j = 0; j < 100_000; j++) c.increment();
            });
            ts[i].start();
        }
        for (Thread t : ts) t.join();
        assertEquals(800_000, c.get());   // fails the build if wrong
    }
}
```

☑ **Lab checkpoint:** `mvn -q verify` must finish with `BUILD SUCCESS` from a clean checkout. Test failures or compile errors fail the build, which is exactly what you want in CI.

Chapter quiz 10 questions

1. Which call actually creates a new thread of execution?

- thread.run()
- thread.start()
- thread.join()
- Thread.yield()

2. A thread that is waiting to enter a synchronized block whose monitor is held by another thread is in state…

- WAITING
- TIMED_WAITING
- BLOCKED
- RUNNABLE

3. What does the volatile keyword guarantee?

- Atomicity of x++
- Visibility and ordering of reads/writes of that field
- Mutual exclusion
- That the field is never cached in the CPU

4. Which is NOT one of the four Coffman conditions required for deadlock?

- Mutual exclusion
- Hold and wait
- Preemption
- Circular wait

5. Object.wait() must be called…

- From any thread at any time
- Only from the main thread
- While holding the monitor of that object (inside synchronized)
- Only on Thread objects

6. What is the default priority of a new Java thread?

- 1 (MIN_PRIORITY)
- 5 (NORM_PRIORITY)
- 10 (MAX_PRIORITY)
- It inherits the OS default, unknown to Java

7. You call worker.interrupt() on a thread that is busy computing and never checks the flag or blocks. What happens?

- It stops immediately
- InterruptedException is thrown in the worker
- Only its interrupted flag is set; it keeps running
- The JVM kills it after a timeout

8. Why is ConcurrentHashMap preferred over Hashtable or Collections.synchronizedMap?

- It is immutable
- It allows null keys
- Fine-grained locking and non-blocking reads give much better concurrency, plus atomic compute/merge
- It preserves insertion order

9. Executors.newVirtualThreadPerTaskExecutor() is the best fit for…

- CPU-bound number crunching
- Thousands of tasks that mostly block on I/O
- Tasks that must run in a strict sequence
- Periodic scheduling

10. Which synchronizer can be reused after it has released the waiting threads?

- CountDownLatch
- CyclicBarrier
- Neither
- Both

WRAP-UP

Summary

- Threads share the heap; each has its own stack – sharing needs synchronisation.
- `start()` creates a thread, `run()` does not. `join()` waits.
- States: NEW → RUNNABLE ⇄ BLOCKED / WAITING / TIMED_WAITING → TERMINATED.
- Races come from shared mutable state + non-atomic compound actions; fix with locks, atomics or immutability.
- `volatile` = visibility, not atomicity.
- Deadlock needs a lock cycle – order your locks or use `tryLock`.
- Use executors and virtual threads instead of raw threads; shut them down properly.
- Cancellation is cooperative: check and restore the interrupt flag.
- Reach for `java.util.concurrent` before writing `wait/notify`.
- Lab-01 is a Maven project: `mvn -q verify` compiles, tests and packages it.

Open Lab-01: 5 tasks + 1 challenge

Next chapter: 02 · Spring Framework